APUNTES UNIVERSITARIOS

Foundations of Machine Learning and Deep Learning

Apuntes del Tema 1: Foundations of Machine Learning — Classical Models & Clustering

18 de octubre de 2026

Índice general

1 Apuntes del Tema 1: Foundations of Machine Learning — Classical Models & Clustering **2**

1.1 1. Conceptos Fundamentales y Paradigmas de Aprendizaje 2

1.1.1 1.1 Trazabilidad y Definición 2

1.1.2 1.2 Paradigmas de Aprendizaje 2

1.2 2. k-Nearest Neighbors (kNN) 2

1.2.1 2.1 Principio e Intuición 2

1.2.2 2.2 Métricas de Distancia 2

1.2.3 2.3 Selección de k y Efectos de Escala 3

1.3 3. Naive Bayes 3

1.3.1 3.1 Fundamento Probabilístico Generativo 3

1.3.2 3.2 Variantes Principales de Naive Bayes 4

1.4 4. Árboles de Decisión (Decision Trees) 4

1.4.1 4.1 Concepto y Particionado Recursivo 4

1.4.2 4.2 Comparativa de Algoritmos y Criterios de División 4

1.5 5. Clustering (Agrupamiento No Supervisado) 5

1.5.1 5.1 k-means 5

1.5.2 5.2 Clustering Jerárquico Aglomerativo (Agglomerative) 5

1.6 6. Práctica y Aplicación en Código PyTorch / Scikit-Learn 5

1 Apuntes del Tema 1: Foundations of Machine Learning — Classical Models & Clustering

1.1 1. Conceptos Fundamentales y Paradigmas de Aprendizaje

1.1.1 1.1 Trazabilidad y Definición

- **Machine Learning:** Disciplina enfocada en la creación de sistemas capaces de realizar tareas mediante el reconocimiento de patrones y la toma de decisiones basadas en datos de experiencia (ejemplos), evaluando el desempeño mediante la capacidad de **generalización** hacia nuevos datos no vistos y previniendo el *overfitting*.
- **Estructura de Datos Básica:**
- **Dataset:** Conjunto de observaciones $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{n}$.
- **Instancia (x):** Vector de características de dimensión d, donde $x \in \mathbb{R}^d$.
- **Etiqueta/Target (y):** Valor objetivo o clase a predecir.
- **Modelo Paramétrico:** Función de mapeo $f(x; \theta) \to \hat{y}$ parametrizada por pesos y sesgos $\theta = (w, b)$.

1.1.2 1.2 Paradigmas de Aprendizaje

- **Supervisado:** Aprendizaje mediante pares entrada-salida etiquetados (x, y). Se divide en:
- **Regresión:** El target y es un valor continuo.
- **Clasificación:** El target y representa una categoría discreta.
- **No Supervisado:** Descubrimiento de estructuras implícitas o agrupamientos en datos sin etiquetas x (ej. *Clustering*, reducción de dimensionalidad).
- **Otros Paradigmas Relevantes:** *Semi-supervised*, *Self-supervised* (pretext tasks), *Transfer Learning* (reutilización de backbones preentrenados), *Active Learning*, *Online/Incremental Learning* y *Federated Learning*.

1.2 2. k-Nearest Neighbors (kNN)

1.2.1 2.1 Principio e Intuición

- **Algoritmo Lazy Learner:** No requiere una fase de entrenamiento o ajuste de parámetros explícito ($f(x)$ se evalúa en tiempo de consulta).
- **Regla de Clasificación:** Asigna la etiqueta a una muestra de prueba x_{query} según el voto mayoritario (o promedio ponderado por la distancia inversa) de sus k vecinos más cercanos en el espacio métrico.

1.2.2 2.2 Métricas de Distancia

- **Distancia Euclídea (L_2):**

$$\text{dist}_{\text{Euclidean}}(x, x') = \sqrt{\sum_{j=1}^{d} (x_j - x'_j)^2}$$

- **Distancia Manhattan (L_1):**

$$\text{dist}_{\text{Manhattan}}(x, x') = \sum_{j=1}^{d} |x_j - x'_j|$$

- **Minkowski (L_p):**

$$\text{dist}_{L^p}(x, x') = \left(\sum_{j=1}^{d} |x_j - x'_j|^p \right)^{1/p}$$

1.2.3 2.3 Selección de k y Efectos de Escala

- **Efecto de k:**
- $k = 1$ (Bajo): Frontera extremadamente flexible, muy sensible al ruido y propenso a sobreajuste (*overfitting*).
- k elevado: Fronteras muy suaves, riesgo de subajuste (*underfitting*) al mezclar vecindarios.
- **Necesidad de Normalización:** Dado que kNN depende de distancias geométricas, las variables con rangos numéricos elevados dominan el cálculo. Es indispensable estandarizar las características ($x' = \frac{x-\mu}{\sigma}$) antes de calcular las distancias.
- **La Maldición de la Dimensionalidad (*Curse of Dimensionality*):** A medida que la dimensión d del espacio crece, el volumen aumenta exponencialmente, volviendo los datos muy dispersos y reduciendo la capacidad discriminativa de las distancias geométricas entre los puntos.

1.3 3. Naive Bayes

1.3.1 3.1 Fundamento Probabilístico Generativo

- **Modelo Generativo:** Modela la probabilidad de la clase a priori $P(y)$ y la verosimilitud de los datos $P(x|y)$ para estimar la probabilidad a posteriori $P(y|x)$ aplicando el Teorema de Bayes:

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)} \propto P(y) \prod_{j=1}^{d} P(x_j|y)$$

- **Regla MAP (Maximum A Posteriori):** Selección de la clase que maximiza el producto:

$$\hat{y} = \arg\max_{y} P(y) \prod_{j=1}^{d} P(x_j|y)$$

- **Asunción "Naive" (Ingenua):** Presupone la **independencia condicional** de todas las características x_j dada la clase y.

1.3.2 3.2 Variantes Principales de Naive Bayes

Variante	Tipo de Variable (x_j)	Estimación de Verosimilitud $P(x_j\|y)$
Categorical NB	Categóricas/Nominales discretas	$P(x_j = v \mid y) = \frac{\text{count}(x_j=v,y)+\alpha}{\text{count}(y)+\alpha\cdot K_j}$
Multinomial NB	Conteos discretos (frecuencias de texto/Bag-of-Words)	$P(x_j \mid y) = \frac{\text{count}(x_j \text{ en clase } y)+\alpha}{\sum_k \text{count}(x_k \text{ en clase } y)+\alpha\cdot V}$
Gaussian NB	Características continuas	$P(x_j \mid y) = \frac{1}{\sqrt{2\pi\sigma_{j,y}^2}} \exp\left(-\frac{(x_j-\mu_{j,y})^2}{2\sigma_{j,y}^2}\right)$

- **Laplace Smoothing ($\alpha = 1$):** Corrige el problema de la **probabilidad cero** (frecuencia nula en datos de entrenamiento que anularía todo el producto de verosimilitudes) sumando el parámetro α en el numerador y $\alpha \cdot K_j$ (o $\alpha \cdot V$) en el denominador.

1.4 4. Árboles de Decisión (Decision Trees)

1.4.1 4.1 Concepto y Particionado Recursivo

- Modelos no paramétricos e interpretables para clasificación y regresión que dividen recursivamente el espacio de características mediante reglas ortogonales a los ejes.

1.4.2 4.2 Comparativa de Algoritmos y Criterios de División

1. **ID3 (Iterative Dichotomiser 3):**

- Solo admite atributos categóricos y no realiza poda.
- Utiliza la **Entropía** $H(S)$ y la **Ganancia de Información** $IG(S, A)$:

```
$$H(S) = - \{}sum_{i=1}^{c} p_i \{}log_2(p_i)$$
```

```
$$IG(S, A) = H(S) - \{}sum_{v \{}in \{}text{Valores}(A)} \{}frac{\{}vert{}S_v\{}vert{}}{\{}
H(S_v)$$
```

1. **C4.5:**

- Soporta variables continuas mediante la evaluación de umbrales binarios de corte t.
- Sustituye el IG por el **Gain Ratio** para corregir el sesgo de ID3 hacia atributos con excesivos valores únicos (como un `ID_Cliente`):

```
$$\{}text{GainRatio}(S, A) = \{}frac{IG(S, A)}{\{}text{SplitInfo}(A)}$$
```

```
$$\{}text{SplitInfo}(A) = -\{}sum_{i=1}^{k} \{}frac{\{}vert{}S_i\{}vert{}}{\{}vert{}S\{}ver
\{}log_2\{}left(\{}frac{\{}vert{}S_i\{}vert{}}{\{}vert{}S\{}vert{}}\{}right)$$
```

- Aplica poda pesimista posterior al entrenamiento.

1. **CART (Classification and Regression Trees):**

- Realiza exclusivamente divisiones binarias.
- Minimiza la **Impureza de Gini** en clasificación:

```
$$G(S) = 1 - \{}sum_{i=1}^{c} p_i^2$$
```

- En regresión, minimiza la varianza/suma de errores cuadráticos (SSE).

1.5 5. Clustering (Agrupamiento No Supervisado)

1.5.1 5.1 k-means

- **Objetivo:** Particionar X en c clusters disjuntos $\omega_1, \ldots, \omega_c$ minimizando la varianza dentro de cada cluster (suma de distancias al cuadrado al centroide μ_i).
- **Paso a Paso del Algoritmo:**

1. Inicialización aleatoria de k centroides.
2. Asignación de cada punto al centroide más cercano.
3. Reordenamiento y recalculo del punto medio/centroide de cada cluster.
4. Repetición iterativa hasta convergencia.

- **Selección de k:** Se evalúa empíricamente mediante el **Elbow Method** (buscando el punto de inflexión en la curva de la inercia/varianza intra-cluster).

1.5.2 5.2 Clustering Jerárquico Aglomerativo (Agglomerative)

- **Mecanismo:** Construye una estructura en árbol (*Dendrograma*) fusionando sucesivamente los dos clusters más cercanos desde un estado inicial donde cada punto es un cluster individual.
- **Criterios de Enlace (Linkage Criteria):**
- **Single Linkage:** Distancia mínima entre miembros de dos clusters ($d_{\min}(C_1, C_2)$).
- **Complete Linkage:** Distancia máxima entre miembros de dos clusters ($d_{\max}(C_1, C_2)$).
- **Average Linkage:** Distancia promedio entre todos los pares de elementos entre clusters.

1.6 6. Práctica y Aplicación en Código PyTorch / Scikit-Learn

```
1
2  import numpy as np
3
4  import torch
5
6  import torch.nn as nn
7
8  import torch.optim as optim
9
10 from sklearn.neighbors import KNeighborsClassifier
11
```

```
from sklearn.naive_bayes import GaussianNB

from sklearn.tree import DecisionTreeClassifier

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler



# ------------------------------------------------------------------------

# 1. Modelos Clásicos en Scikit-Learn (kNN, Naive Bayes, Decision Trees)

# ------------------------------------------------------------------------

def train_classical_models(X_train, y_train, X_test):

    # kNN: requiere estandarización previa de características

    scaler = StandardScaler()

    X_train_scaled = scaler.fit_transform(X_train)

    X_test_scaled = scaler.transform(X_test)



    knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')

    knn.fit(X_train_scaled, y_train)



    # Gaussian Naive Bayes

    gnb = GaussianNB()

    gnb.fit(X_train, y_train)



    # Árbol de Decisión CART (Criterio Gini)

    tree = DecisionTreeClassifier(criterion='gini', max_depth=5)

    tree.fit(X_train, y_train)



    return knn, gnb, tree



# ------------------------------------------------------------------------

# 2. Clustering (k-means)

```

```
# ------------------------------------------------------------------------

def run_kmeans_clustering(X_data, n_clusters=3):

    kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)

    cluster_labels = kmeans.fit_predict(X_data)

    centroids = kmeans.cluster_centers_

    return cluster_labels, centroids



# ------------------------------------------------------------------------

# 3. MLP Baseline con Vectorización (Aplanado) en PyTorch

# ------------------------------------------------------------------------

class MLPClassifier(nn.Module):

    """

    MLP Baseline para imágenes o datos vectorizados.

    Demuestra la pérdida de estructura espacial al aplanar un tensor (C, H, W) -> (C * H * W)

    """

    def __init__(self, input_shape=(3, 64, 64), num_classes=10, hidden_units=[512, 256], dropout_rate=0.3):

        super(MLPClassifier, self).__init__()



        # Dimensión de la entrada aplanada

        self.input_dim = input_shape[0] * input_shape[1] * input_shape[2]



        layers = []

        in_dim = self.input_dim



        for h_dim in hidden_units:

            layers.append(nn.Linear(in_dim, h_dim))

            layers.append(nn.ReLU())

            layers.append(nn.Dropout(p=dropout_rate))

```

```
            in_dim = h_dim


        # Capa lineal de salida (retorna logits para CrossEntropyLoss)
        layers.append(nn.Linear(in_dim, num_classes))


        self.network = nn.Sequential(*layers)


    def forward(self, x):
        # Aplanado explícito: (Batch, Channels, Height, Width) -> (Batch, C * H * W)
        x = x.view(x.size(0), -1)
        return self.network(x)


if __name__ == "__main__":
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    dummy_images = torch.randn(16, 3, 64, 64).to(device)
    model = MLPClassifier(input_shape=(3, 64, 64), num_classes=10).to(device)
    logits = model(dummy_images)
    print(f"Forma de la entrada : {dummy_images.shape}")
    print(f"Forma de los logits : {logits.shape}")
```